

MODULE II - Agile Process

Agility and cost of change, agility principles, politics of agile development, Human factors, Extreme Programming, Adaptive software development (ASD), SCRUM, Kanban, Feature Driven Programming, Lean Software Development, Test Driven Development.

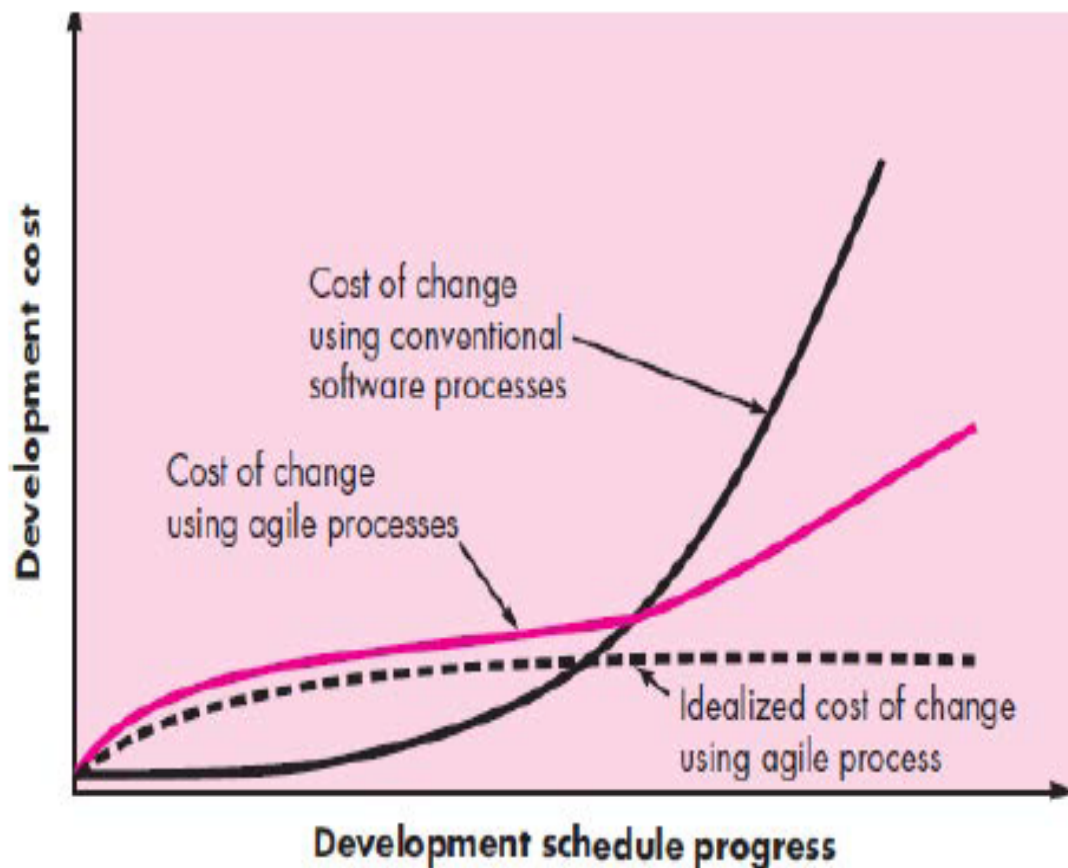
INTRODUCTION:

Agile software engineering has a set of development guidelines. The mechanism encourages customer satisfaction and early incremental delivery of software.

- Software engineers and other project stakeholders (managers, customers, end users) work together on an agile team.
- Agile software engineering represents a reasonable alternative to conventional software engineering.

Agility:

- Agility means effective (rapid and adaptive) response to change, effective communication among all stakeholder.
- Drawing the customer onto team and organizing a team so that it is in control of work performed. The Agile process is light-weight methods and People-based rather than plan-based methods.
- The agile process forces the development team to focus on software itself rather than design and documentation.
- The agile process believes in iterative method.
- The aim of agile process is to deliver the working model of software quickly to the customer For example: Extreme programming is the best known of agile process.



1. AGILITY AND THE COST OF CHANGE

The cost of change increases as the project progresses. It is relatively easy to accommodate a change when a software team is gathering requirements (early in a project). The team is in the middle of validation testing (something that occurs relatively late in the project), and stakeholder is requesting a major functional change. The change requires a modification to the architectural design of the software, the design and construction of three new components, modifications to another five components, the design of new tests, and so on. Costs and time increases quickly. A well-designed agile process “flattens” the cost of change curve, shaded, solid curve), allowing a software team to accommodate changes late in a software project without cost and time impact. the agile process encompasses incremental delivery.

What is an Agile Process:

Any agile software process is characterized in a manner that addresses a number of key assumptions about the majority of software projects:

1. It is difficult to predict (estimate for future change) in advance which software requirements will persist and which will change.
2. For many types of software, design and construction are performed simultaneously. It is difficult to predict how much design is necessary before

construction is used to prove the design.

3. Analysis, design, construction, and testing are not expected. (from a planning point of view)

1.1 Agility Principles

12 agility principles for those who want to achieve agility:

1. Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late in development.
3. Deliver working software frequently, from a couple of weeks to a couple of Months.
4. Stakeholders and software engineers must work together daily throughout the project.
5. Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
7. Working software is the primary measure of progress.
8. Agile processes promote sustainable development.
9. Continuous attention to technical excellence and good design enhances (increases) agility.
10. Simplicity—the art of maximizing the amount of work not done—is essential.
11. The best architectures, requirements, and designs emerge from self-organizing teams.
12. At regular intervals, the team reflects on how to become more effective and then adjusts its behavior accordingly.

1.2 The Politics of Agile Development

There is considerable debate (sometimes strident) about the benefits and applicability of agile software development as opposed to more conventional software engineering processes. Jim Highsmith [Hig02a] (facetiously) states the extremes when he characterizes the feeling of the pro-agility camp (“agilists”). “Traditional methodologists are a bunch of stick-in-the-muds who’d rather produce flawless documentation than a working system that meets business needs.” As a counterpoint, he states (again, facetiously) the position of the traditional software engineering camp: “Lightweighters, ‘agile’ methodologists are a bunch of glorified hackers who are going to be in for a heck of a surprise when they try to scale up their toys into enterprise-wide software.”

Like all software technology arguments, this methodology debate risks degenerating into a religious war. If warfare breaks out, rational thought disappears and beliefs rather than facts guide decision making. No one is against agility. The real question is: What is the best way to achieve it?

As important, how do you build software that meets customers’ needs today and exhibits the quality characteristics that will enable it to be extended and scaled to meet customers’ needs over the long term? There are no absolute answers to either of these questions. Even within the agile school itself, there are many proposed process models each with a subtly different approach to the agility problem. Within each

model there is a set of “ideas” (agilists are loath to call them “work tasks”) that represent a significant departure from traditional software engineering.

And yet, many agile concepts are simply adaptations of good software engineering concepts. Bottom line: there is much that can be gained by considering the best of both schools and virtually nothing to be gained by denigrating either approach.

1.3 Human Factors

Agile software development take the importance of “people factors”. Number of different talents must exist among the people on an agile team and the team itself:

Competence: “competence” encompasses talent, specific software-related skills, and overall knowledge of the process .

Common focus: Members of the agile team may perform different tasks and all should be focused on one goal—to deliver a working software increment to the customer within the time promised.

Collaboration: Team members must collaborate with one another and all other Stakeholders to complete the their task.

Decision-making ability: Any good software team (including agile teams) must be allowed the freedom to control its own destiny.

Fuzzy problem-solving ability: The agile team will continually have to deal with ambiguities(confusions or doubts).

Mutual trust and respect: The agile team exhibits the trust and respect .

Self-organization:

- (1) the agile team organizes itself for the work to be done
- (2) the team organizes the process to best accommodate its local environment
- (3) the team organizes the work schedule to best achieve delivery of the software increment.

1.4 Extreme Programming:

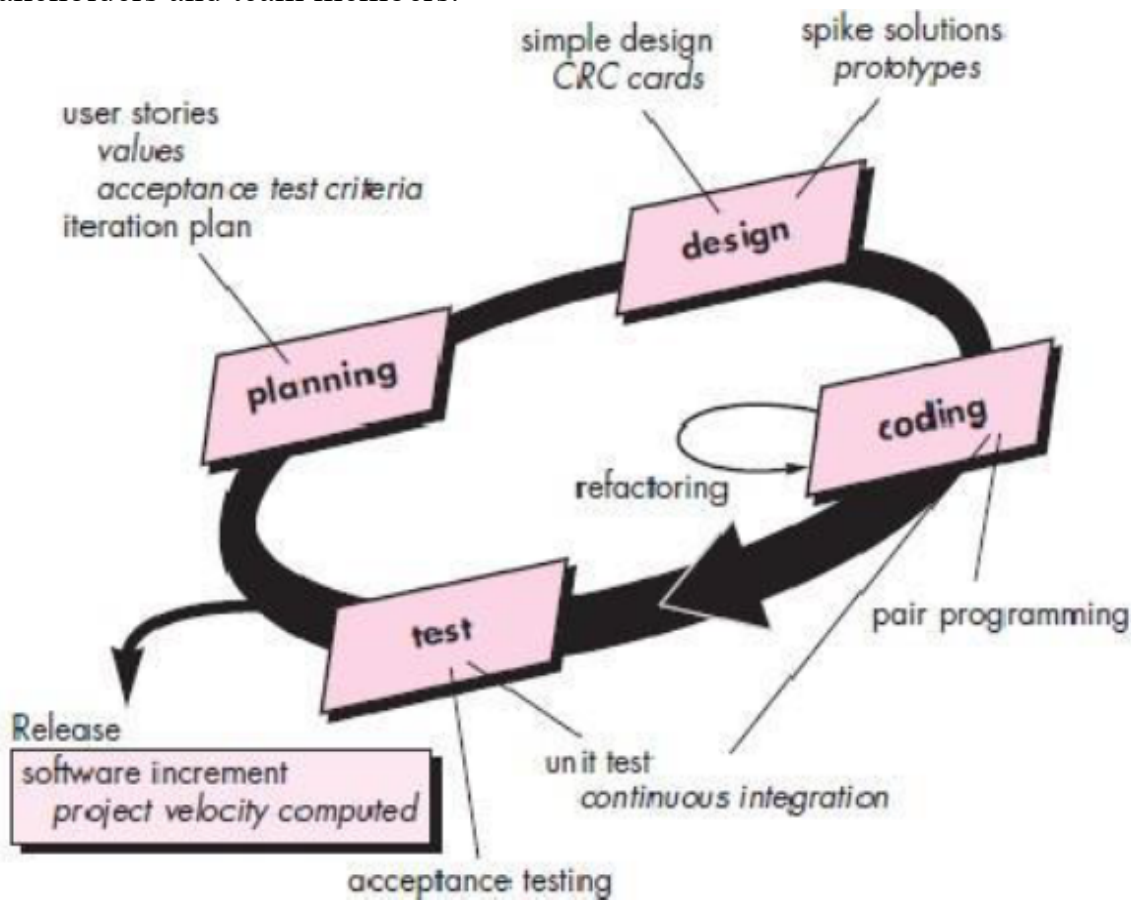
Extreme Programming (XP), the most widely used approach to agile software development. XP proposed by kent beck during the late 1980’s.

XP Values

Beck defines a set of five *values* —communication, simplicity, feedback, courage, and respect. Each of these values is used in XP activities, actions, and tasks.

- ☐ Effective **communication** between software engineers and other stakeholders .
- ☐ To achieve **simplicity**, XP restricts developers to design only for immediate needs, rather than future needs.
- ☐ **Feedback** is derived from three sources: the implemented software itself, the customer, and other software team members

- **courage.(discipline)** An agile XP team must have the discipline (courage) to design for today, recognizing that future requirements may change dramatically.
- the agile team inculcates **respect** among its members, between other stakeholders and team members.



The XP(Extreme Programming) Process

XP Process have four framework activities: planning, design, coding, and testing.

Planning. The planning activity begins with *listening*—a requirements gathering activity.

- Listening leads to the creation of a set of “stories” (also called *user stories*) that describe required output, features, and functionality for software to be built.
- Each *story* is written by the customer and is placed on an index card. The customer assigns a *value* (i.e., a priority) to the story based on the overall business value of the feature or function.
- Members of the XP team then assess each story and assign a *cost*—measured in development weeks—to it.
- If the story is estimated to require more than three development weeks, the story is broken into smaller stories and the assignment of value and cost occurs again. It is important to note that new stories can be written at any time.
- the stories with highest value will be moved up in the schedule and implemented first.

Design: XP design follows the KIS (keep it simple) principle.

- If a difficult design problem is encountered as part of the design of a story,

XP recommends the immediate creation of an operational prototype of that portion of the design. Called a *spike solution*.

- ☐ XP encourages *refactoring*—a construction technique that is also a method for design optimization.
- ☐ Refactoring is the process of changing a software system in a way that it does not change the external behavior of the code and improves the internal structure.

Coding: Design work is done, the team does *not* move to code, develops a series of unit tests for each of the stories that is to be included in the current release (software increment).

- ☐ Once the unit test has been created, the developer is better able to focus on what must be implemented to pass the test.
- ☐ Once the code is complete, it can be unit-tested immediately, and providing feedback to the developers.
- ☐ A key concept during the coding activity is *pair programming*. i.e. two people work together at one computer workstation to create code for a story.
- ☐ As pair programmers complete their work, the code they develop is integrated with the work of others.

Testing:

- ☐ As the individual unit tests are organized into a “universal testing suite” integration and validation testing of the system can occur on a daily basis.
- ☐ XP *acceptance tests*, also called *customer tests*, are specified by the customer and focus on overall system features and functionality.

Acceptance tests are derived from user stories that have been implemented as part of a software release.

Industrial XP

Joshua Kerievsky describes *Industrial Extreme Programming* (IXP)

IXP has six new practices that are designed to help XP process works successfully for projects within a large organization.

Readiness assessment. the organization should conduct a *readiness assessment*.

- (1) Development environment exists to support IXP(Industrial Extreme Programming).
- (2) the team will be populated by the proper set of stakeholders.
- (3) the organization has a distinct quality program and supports continuous improvement.
- (4) the organizational culture will support the new values of an agile Team.

Project community.

- ☐ Classic XP suggests that the right people be used to populate the agile team to ensure success.
- ☐ The people on the team must be well-trained, adaptable and skilled.
- ☐ A community may have a team members and customers who are central to the success of the project.

Project chartering.:

- Chartering examines the context of the project to determine how it complements, extends, or replaces existing systems or processes.

Test-driven management.

- Test-driven management establishes a series of measurable “destinations” and then defines mechanisms for determining whether or not these destinations have been reached.

Retrospectives.

- An IXP team conducts a specialized technical reviews after a software increment is delivered. Called a *retrospective*.
- The review examines “issues, events, and lessons-learned” across a software increment and/or the entire software release.

Continuous learning.

- learning is a vital part of continuous process improvement, members of the XP team are encouraged to learn new methods and techniques that can lead to a higher quality product.

The XP Debate

Requirements volatility. The customer is an active member of the XP team, changes to requirements are requested informally. As a consequence, the scope of the project can change and earlier work may have to be modified to accommodate current needs.

- **Conflicting customer needs.** Many projects have multiple customers, each with his own set of needs.
- **Requirements are expressed informally.** User stories and acceptance tests are the only explicit manifestation of requirements in XP. specification is often needed to remove inconsistencies, and errors before the system is built.
- **Lack of formal design:** When complex systems are built, design must have the overall structure of the software then it will exhibit quality.

1.5 Adaptive software development (ASD)

Adaptive Software Development (ASD) has been proposed by Jim Highsmith [Hig00] as a technique for building complex software and systems. The philosophical underpinnings of ASD focus on human collaboration and team self-organization. Highsmith argues that an agile, adaptive development approach based on collaboration is “as much a source of *order* in our complex interactions as discipline and engineering.” He defines an ASD “life cycle” that incorporates three phases, speculation, collaboration, and learning.

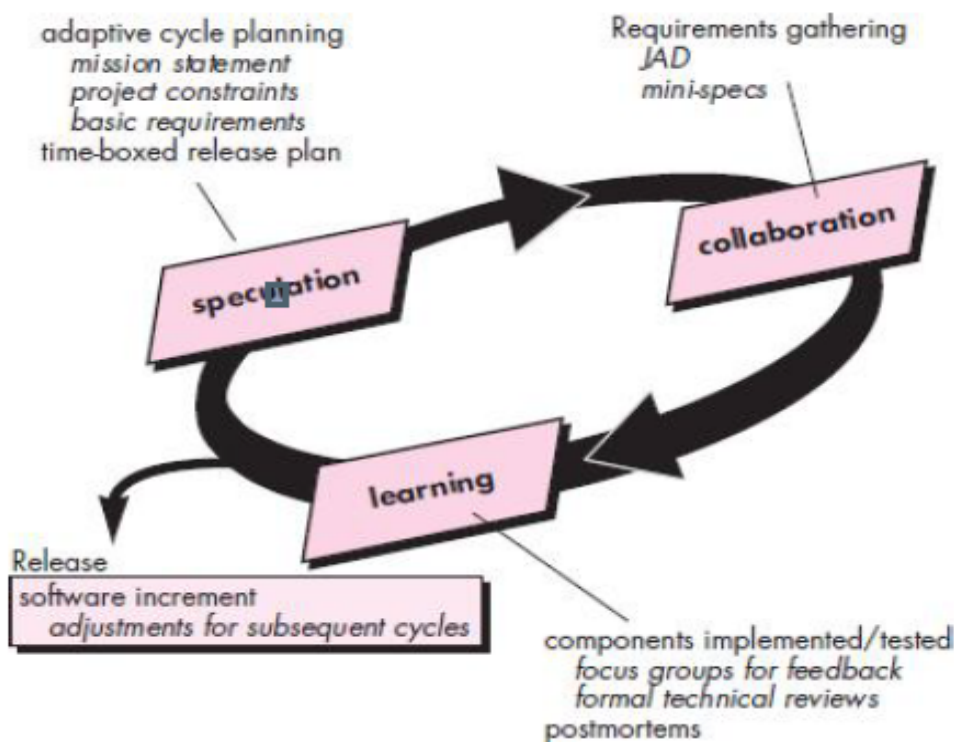
During *speculation*, the project is initiated and *adaptive cycle planning* is conducted. Adaptive cycle planning uses project initiation information—the customer’s mission statement, project constraints (e.g., delivery dates or user descriptions), and basic requirements—to define the set of release cycles (software increments) that will be required for the project.

No matter how complete and farsighted the cycle plan, it will invariably change. Based on information obtained at the completion of the first cycle, the plan is reviewed and adjusted so that planned work better fits the reality in which an ASD team is working.

Motivated people use *collaboration* in a way that multiplies their talent and creative output beyond their absolute numbers. This approach is a recurring theme in all agile methods. But collaboration is not easy. It encompasses communication and teamwork, but it also emphasizes individualism, because individual creativity plays an important role in collaborative thinking.

It is, above all, a matter of trust. People working together must trust one another to

- (1) criticize without animosity,
- (2) assist without resentment,
- (3) work as hard as or harder than they do,
- (4) have the skill set to contribute to the work at hand, and
- (5) communicate problems or concerns in a way that leads to effective action.



1.6 SCRUM

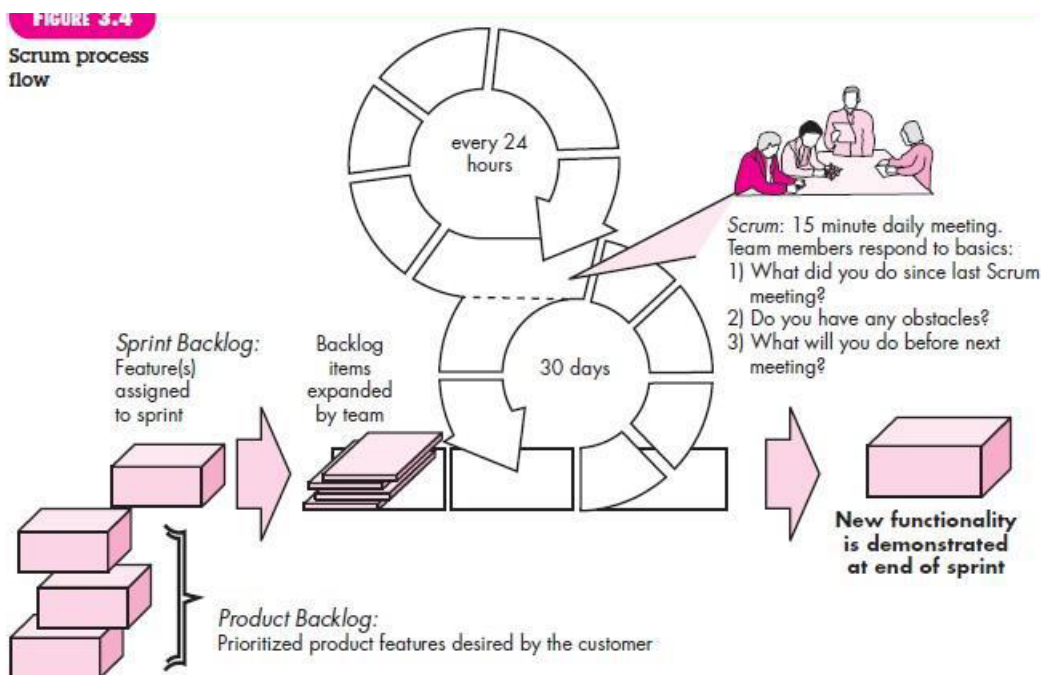
Scrum is an agile software development method that was coined by Jeff Sutherland and his development team in the early 1990's.

- Scrum has the following framework activities: requirements, analysis, design, evolution, and delivery. Within each framework activity, actions and work tasks occur within a process called a *sprint*.
- scrum defines a set of development actions:

Backlog—a prioritized list of project requirements or features that provide business value for the customer. Items can be added to the backlog at any time (this is how changes are introduced). The product manager assesses the backlog and updates priorities as required.

Sprints—consist of work units that are required to achieve a requirement defined in the backlog .

Scrum



Scrum meetings—are short (typically 15 minutes) meetings held daily by the Scrum team. Three key questions are asked and answered by all team members.

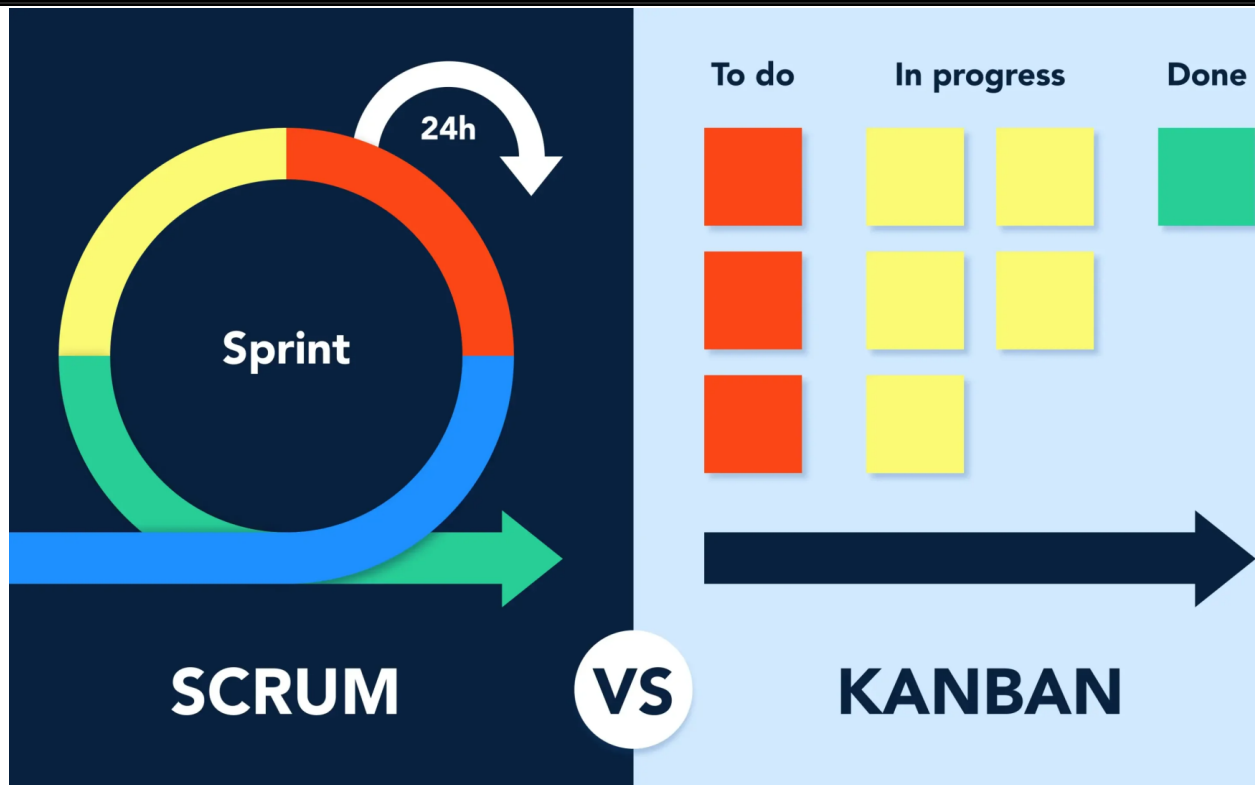
- What did you do since the last team meeting?
- What obstacles are you encountering?
- What do you plan to accomplish by the next team meeting?

A team leader, called a *Scrum master*, leads the meeting and assesses the responses from each person.

Demos—deliver the software increment to the customer so that functionality that has been implemented can be demonstrated and evaluated by the customer.

1.7 KANBAN

- Kanban and Scrum are both popular frameworks within Agile project management, but they differ in approach
- Scrum uses fixed-length, time-boxed sprints to deliver work incrementally with defined roles and events.
- while Kanban is a continuous, flow-based system that visualizes workflow, limits work-in-progress (WIP), and focuses on optimizing the flow of value through the process.
- While Scrum provides more structure and predictability, Kanban offers greater flexibility and is ideal for processes with a continuous flow of work and frequently changing priorities.



1.8 Feature Driven Programming

Feature Driven Development (FDD) was originally coined by Peter Coad and his colleagues as a process model for object-oriented software engineering.

- A *feature* “is a client-valued function that can be implemented in two weeks or less”.
- the definition of features provides the following benefits:
 - Features are small blocks of deliverable functionality, users can describe them more easily.
 - Since a feature is the FDD deliverable software increment, the team develops operational features every two weeks.
 - Because features are small, their design and code representations are easier.

- The following template for defining a feature:

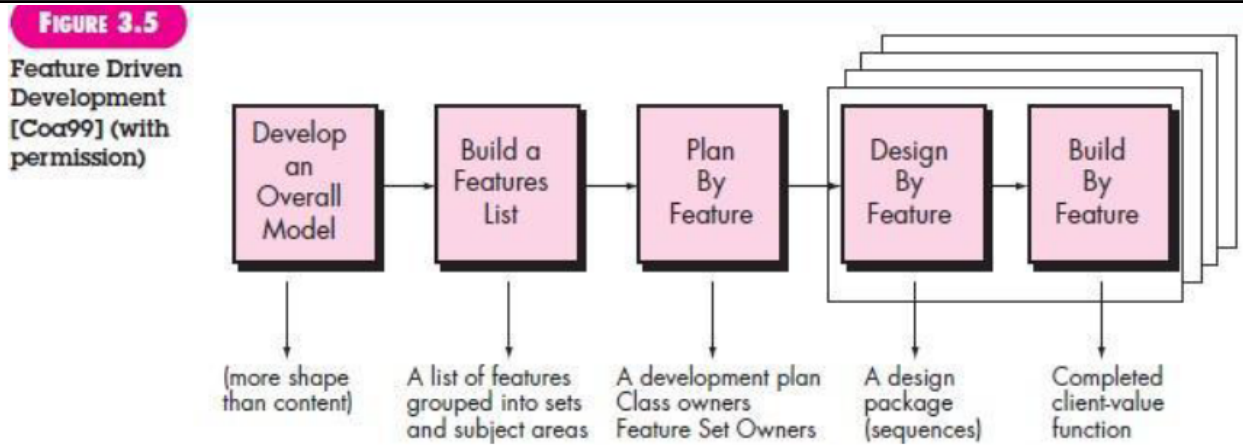
<action> the <result> <by | for | of | to> a(n) <object>

Where

<object> is “a person, place, or thing.”

Examples of features for an **e-commerce application** might be:

- 1) *Add the product to shopping cart*
- 2) *Store the shipping-information for the customer*



A feature set groups related features into business-related categories and is defined as:

<action><-ing> a(n) <object>

For example: *Making a product sale* is a feature set that would encompass the features noted earlier and others.

- It is essential for developers, their managers, and other stakeholders to understand project status.
- For that FDD defines six milestones during the design and implementation of a feature: “design walkthrough, design, design inspection, code.

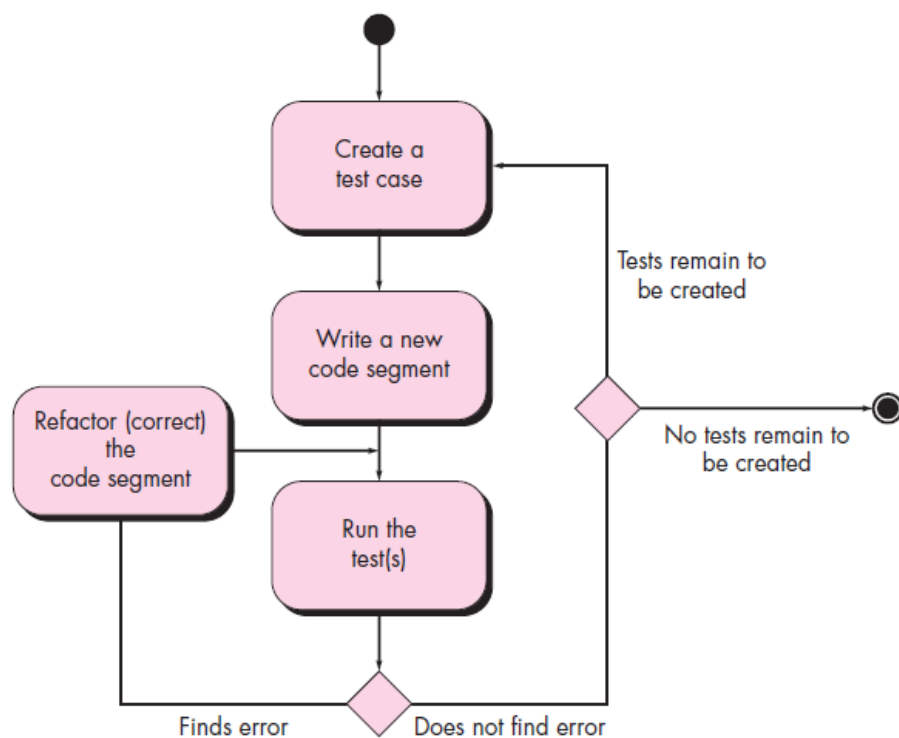
1.9 Lean Software Development (LSD)

- *Lean Software Development (LSD)* has adapted the principles of lean manufacturing to the world of software engineering.
- LSD process can be summarized as *eliminate waste, build quality in, create knowledge, defer commitment, deliver fast, respect people, and optimize the whole*.
- For example,
eliminate waste within the context of an agile software project as
 - 1) adding no extraneous features or functions
 - (2) assessing the cost and schedule impact of any newly requested requirement,
 - (3) removing any superfluous process steps,
 - (4) establishing mechanisms to improve the way team members find information,
 - (5) ensuring the testing finds as many errors as possible,

1.10 Test-Driven Development

- During TDD, code is developed in very small increments (one subfunction at a time), and no code is written until a test exists to exercise it.
- You should note that each iteration results in one or more new tests that are added to a regression test suite that is run with every change.
- This is done to ensure that the new code has not generated side effects that cause errors in the older code.

FIGURE 31.3
Test-driven
development
process flow



Reference: Software Engineering: A Practitioner's Approach, Roger S. Pressman, Bruce R. Maxim, 9th Edition, McGraw Hill Education, 2024

